

فصل سوم

مقدمه

ساده‌ترین نوع کارگزار که تا اینجا شناختیم، کارگزار واکنشی بود که بر اساس نگاشتی مستقیم از حالت به عمل تصمیم‌گیری می‌کرد. این نوع کارگزار برای محیط‌های پیچیده و بزرگ مناسب نیست، چراکه یادگیری و ذخیره‌سازی نگاشت‌ها در این محیط‌ها بسیار زمان‌بر است. برای چنین محیط‌هایی، کارگزارهای مبتنی بر هدف عملکرد بهتری دارند.

در این فصل با نوعی از این کارگزارها تحت عنوان **کارگزار حل مسئله** آشنا می‌شویم. این دسته از الگوریتم‌ها "ناآگاهانه" هستند، به این معنا که هیچ دانشی از مسئله به‌جز تعریف آن ندارند.

کارگزارهای حل مسئله

اولین گام در حل مسئله، تعیین یک **هدف** بر اساس حالت فعلی و **مقیاس کارایی** کارگزار است. وظیفه‌ی کارگزار آن است که رشته‌ای از اقدامات را پیدا کند که وی را به حالت هدف برساند.

تدوین مسئله، فرایند تصمیم‌گیری درباره‌ی انتخاب حالت‌ها و اقدامات بر اساس یک هدف خاص است. اگر کارگزار اطلاعاتی بیش از تعریف مسئله نداشته باشد، ناچار به انتخاب تصادفی خواهد بود.

روش تصمیم‌گیری کارگزار، بررسی توالی‌های مختلفی از اقدامات است که به حالت‌هایی با ارزش معلوم ختم می‌شوند، و سپس انتخاب بهترین توالی ممکن است. به این فرایند، **جستجو** گفته می‌شود.

الگوریتم جستجو، مسئله را به‌عنوان ورودی دریافت می‌کند و یک **رامحل** به‌شکل رشته‌ای از اقدامات باز می‌گرداند. پس از یافتن رامحل، می‌توان آن را **اجرا** کرد. بنابراین کارگزار حل مسئله از سه بخش تشکیل می‌شود: **تدوین**، **جستجو**، **اجرا**.

در طراحی این نوع کارگزار، فرض می‌شود محیط:

- ایستا
- رویت‌پذیر
- گسسته
- معین باشد

از آنجا که رامحل‌ها دنباله‌ای ساده از اقدامات اند، نمی‌توانند با رویدادهای غیر منتظره مقابله کنند. همچنین کارگزار در مرحله‌ی اجرا بدون دریافت بازخورد از محیط، رامحل را اجرا می‌کند؛ بنابراین باید از موفقیت آن اطمینان داشته باشد.

فرایند تدوین مسئله

یک مسئله از چهار مؤلفه‌ی اصلی تشکیل می‌شود:

1. **حالت ابتدایی:** نقطه شروع جستجو.
2. **تابع پسین:** تعیین اقدامات ممکن از یک حالت معین. توالی حالت‌ها که از طریق اعمال این توابع به یکدیگر متصل می‌شوند، فضای حالت را تشکیل می‌دهند.
3. **آزمون هدف:** مشخص می‌کند که آیا یک حالت خاص، حالت هدف هست یا نه.
4. **تابع هزینه مسیر:** به هر مسیر یک مقدار عددی تخصیص می‌دهد. این تابع نمایانگر مقیاس کارایی کارگزار است.

در این فرایند، **چکیده‌سازی (Abstraction)** برای ساده‌سازی مسئله ضروری است، چراکه در غیر این‌صورت کارگزار در جزئیات مسئله گم می‌شود.

دو نوع مدل تدوین مسئله وجود دارد:

- **تدوین افزایشی:** با یک حالت توصیف‌نشده آغاز می‌کند و به تدریج آن را گسترش می‌دهد.
- **تدوین کامل:** با حالت کامل آغاز می‌کند و با هر اقدام، آن را تغییر می‌دهد.

فرایند حل مسئله

حل مسئله از طریق **جستجو در فضای حالت** انجام می‌شود. در این فصل، از **درخت جستجوی صریح** استفاده شده است. این درخت با استفاده از حالت اولیه و تابع پسین (که با هم فضای حالت را تعریف می‌کنند) ساخته می‌شود.

- ریشه درخت جستجو، گره‌ای است که با حالت اولیه متناظر است.
- ابتدا بررسی می‌شود آیا این گره حالت هدف است یا خیر.
- اگر نباشد، گره گسترش می‌یابد: تابع پسین روی آن اعمال شده و حالت‌های جدیدی ایجاد می‌شود.
- این‌که از بین حالت‌های گسترش‌یافته کدام را انتخاب کنیم یا به عقب برگردیم، به **راهبرد جستجو** بستگی دارد.

تفاوت فضای حالت و درخت جستجو: فضای حالت فقط حالت‌ها را نگه می‌دارد، در حالی که درخت جستجو ساختار سلسله‌مراتبی گره‌ها را شامل می‌شود.

هر گره دارای اطلاعات زیر است:

- **حالت:** حالتی از فضای حالت که با گره متناظر است
- **گره والد:** گره‌ای که این گره را تولید کرده
- **اقدام:** عملی که منجر به تولید این گره از گره والد شده
- **هزینه مسیر $g(n)$:** هزینه طی‌شده تا رسیدن به این گره
- **عمق:** تعداد اقدامات از حالت ابتدایی تا این گره

مجموعه لبه (Frontier): مجموعه‌ای از گره‌ها که تولید شده‌اند اما هنوز گسترش نیافته‌اند. راهبرد جستجو، گره بعدی برای گسترش را از این مجموعه انتخاب می‌کند.

اندازه گیری کارایی حل مسئله

خروجی الگوریتم می‌تواند یک رامحل یا شکست باشد. کارایی آن از چهار زاویه ارزیابی می‌شود:

1. کامل بودن: آیا الگوریتم همیشه جواب را می‌یابد؟
2. بهینگی: آیا همیشه بهترین جواب را پیدا می‌کند؟
3. پیچیدگی زمانی: زمان لازم برای یافتن جواب؟
4. پیچیدگی فضایی: میزان فضای لازم برای انجام جستجو؟

پیچیدگی زمانی و فضایی بر اساس سختی مسئله و اندازه گراف فضای حالت بررسی می‌شود.

سه پارامتر مهم در ارزیابی پیچیدگی:

- b: ضریب انشعاب (تعداد بیشینه‌ی پسین‌های هر گره)
- d: عمق کم عمق‌ترین جواب
- m: طولانی‌ترین مسیر ممکن در فضای حالت

برای برآورد کارایی می‌توان:

- فقط هزینه جستجو را در نظر گرفت
- یا هزینه کلی (هزینه جستجو + هزینه مسیر بهینه)

راهبرد های جستجوی ناآگاهانه

این راهبرد ها جستجوی کور هستند، این راهبرد ها تنها قادر به تولید پسین ها و تشخیص هدف از یک حالت غیر هدف هستند. راهبرد هایی که میدانند یک حالت غیر هدف، بخت موفقیت بیشتری دارد جستجوی آگاهانه یا آروینی است که فصل بعد

جستجوی اول سطح

راهبردی ساده است که در آن ابتدا ریشه گسترش میابد ، سپس تمام پسین های آن گره ریشه ، به طور کلی باید تمام پسین های یک سطح گسترش یابد قبل اینکه به سراغ بعدی برویم. جستجوی اول سطح را میتوانیم با فراخوانی tree-search با یک لبه خالی که یک صف FIFO است پیاده سازی کنیم

توی صف FIFO گره هایی که زودتر وارد میشوند زودتر گسترش خواهند یافت

این جستجو یک جستجوی کامل است

جستجوی اول سطح همواره راهبرد مطلوب انتخاب نمیشود چرا که از لحاظ پیچیدگی زمانی و فضایی آن بسیار بالا $O(b^d)$ میشود.

مشکلی که دارد در الگوریتم جستجوی اول سطح نیاز به حافظه بالایی دارد ، همچنین نیاز به زمان زیادی دارد به طور کلی مسائل با پیچیدگی نمایی به جز نمونه های کوچک ، توسط نمونه های ناآگاهانه قابل حل نیستند.

جستجوی هزینه یکنواخت

جستجوی اول سطح زمانی بهینه است که هزینه تمام مراحل یکسان باشد، در این روش به جای گسترش کم عمق ترین گره ، گره n را که کمترین هزینه مسیر را دارد گسترش می دهد

اگر تمامی مراحل هزینه یکسانی داشته باشند این جستجو با جستجوی اول سطح یکسان است

جستجوی هزینه یکنواخت، به تعداد مراحل اهمیت نمیدهد و تنها هزینه کل برای آن مهم است ولی ممکن است به مشکل بخوریم چرا که اگر به مرحله ای برسیم که با هزینه صفر به خودش برگردد در یک لوپ بی نهایت گیر میکند

جستجوی اول عمق

جستجوی اول عمق همیشه عمیق ترین گره را از لبه فعلی درخت جستجو گسترش می دهد، این جستجو بوسیله همان tree-search به همراه یک پشته پیاده سازی میشود
جستجوی اول عمق به حافظه کمتری نیاز دارد، این مسیر تنها نیاز دارد که یک مسیر از ریشه تا گره برگه ، همراه با گره های گسترش نیافته تمام آن مسیر نگه داری کند
درمورد فضای حالتی با ضریب انشعاب b حداکثر عمق m جستجوی اول عمق به ذخیره سازی تنها $bm+1$ گره نیاز دارد

نوع خاصی از جستجوی اول عمق به نام جستجوی پس گرد وجود دارد که از حافظه کمتری استفاده می کند در این نوع به خاطر میسپارد که کدام پسین را باید در مرحله بعد تولید کند اینطور بجای $O(bm)$ به $O(m)$ حافظه نیاز دارد

مشکل جستجوی اول عمق این است که ممکن است انتخاب غلطی انجام بدیم و در پایین رفتن از یک مسیر بسیار طولانی و حتی نامتناهی گیر کند ، در حالی که یک انتخاب دیگر می توانست به پیدا کردن جوابی در نزدیکی ریشه درخت جستجو منجر شود، بنابراین جستجوی اول عمق بهینه نیست .

جستجوی عمق محدود

مشکل درختان نامحدود را می توان با در نظر یک محدودیت عمق از پیش تعیین شده برطرف کرد ، به این جستجو جستجوی عمق محدود می گویند. این جستجو مشکل مسئله مسیر نامتناهی را حل میکند ، اما اگر متاسفانه محدودیت عمق کمتر از عمق جواب باشد به عامل ناتمامی میرسیم

معمولا عمق با توجه به مسئله مشخص میشود، برای مثال در نقشه رومانی اگر بدانیم ۲۰ شهر داریم محدودیت عمق را ۱۹ میگذاریم ولی خب تو بیشتر مسائل تا حل نکنیم چنین اطلاعاتی بدست نخواهیم آورد

جستجوی اول عمق عمیق شونده تکراری

یک راهبرد عمومی است که معمولاً با جستجوی اول عمق استفاده می‌شود و بهترین محدودیت عمق را می‌یابد. این کار با افزایش تدریجی محدودیت تا زمانی که یک هدف پیدا شود انجام دهد. این روش مزیت های جستجوی اول عمق و اول سطح را یکجا جمع می‌کند. همانند جستجوی اول عمق از حافظه نسبتاً کمی استفاده می‌کند. توی این روش با محدودیت عمق بررسی میکنیم و اگر پیدا نشد عمق مجاز را افزایش داده و دوباره بررسی میکنیم.

ممکنه این الگوریتم اسراف کارانه بنظر بیاد چون که حالت ها چند بار تولید میشوند ولی اینطور نیست چرا که گره‌های نزدیک به ریشه بارها بررسی می‌شن، ولی تعدادشون کمه ، گره‌های دورتر تعدادشون زیاده، ولی فقط یکبار بررسی می‌شن. و با این علی رغم تولید تکراری حالت ها سریع تر از جستجوی اول سطح است. به طور کلی زمانی که فضای جستجو بزرگ و عمق راه حل نامعلوم است، جستجوی عمیق شونده تکراری روش جستجوی ناآگاهانه مطلوب می باشد.

الگوریتم دیگری که بجای محدودیت عمق در حال افزایش ، از محدودیت هزینه مسیر در حال افزایش استفاده کنیم الگوریتم حاصل شده جستجوی طولانی کننده تکراری نامیده می شود این الگوریتم مشخص شده که نسبت به جستجوی طولانی کننده تکراری سربار قابل توجهی دارد

جستجوی دوطرفه

اساس آن انجام دو جستجو همزمان می باشد. یه جستجو از مبدا به سمت هدف می‌ره و کی دیگه از هدف به سمت مبدا انگیزه استفاده از این روش این است که مقدار $b^2d + b^2d$ بسیار کوچک تر از b^d است

در این روش یک یا هر دو این دو جستجو ها قبل از گسترش گره بررسی میکند که آیا آن گره در لبه درخت جستجو دیگر قرار دارد یا نه ، اگر چنین بود یعنی یک راه حل پیدا شده

اگر هر دو جستجو از نوع اول سطح باشند ، الگوریتم کامل و بهینه است

اگر چند تا هدف مختلف داریم ، می‌تونیم یه "هدف جدید خیالی" تعریف کنیم این هدف خیالی طوری طراحی می‌شه که پیش‌نیاز یا پدرهای مستقیمش، همون هدف‌های واقعی باشن. اینطوری جستجو به جای اینکه به چند تا نقطه جداگانه ختم بشه، فقط دنبال یه نقطه واحد می‌گرده! با این حرکت گره‌های تکراری یا مسیرهای اضافی را حذف میکنیم

بدترین حالت اینه که ندونی دقیقاً هدف‌هات چیه، بلکه فقط بدونی چه شرایطی باید داشته باشند.

مقایسه راهبرد های جستجوی ناآگاهانه

راهبرد ها را براساس چهار معیار ارزیابی با هم مقایسه می کنند

	دوطرفه	عمیق شونده تکراری	با عمق محدود	اول عمق	هزینه یکنواخت	اول سطح
کامل ؟	بله	بله	بله (اگر عمق زیر عمق جواب باشد)	نه	بله	بله
زمان	$O(b^{(d/2)})$	$O(b^d)$	$O(b^l)$ عمق محدود = l	$O(b^m)$ عمق بیشینه = m	$O(b^{(1 + LC*/\epsilon)})$	$O(b^d)$
فضا	$O(b^{(d/2)})$	$O(bm)$	$O(bl)$	$O(bm)$	$O(b^{(1 + LC*/\epsilon)})$	$O(b^d)$
بهینه ؟	بله	بله	نه (اگر جواب در عمق بیشتر باشد ممکن پیدا نشه)	نه	بله	بله (در هزینه های یکنواخت)

کامل ؟ : آیا همیشه یک جواب پیدا می‌کند اگر جواب وجود داشته باشد؟

زمان : چند تا گره باید تولید/بررسی شوند؟

فضا : چقدر حافظه نیاز دارد؟

بهینه بودن : آیا بهترین جواب (با کمترین هزینه) را می‌دهد؟

اجتناب از حالت های تکراری

تا اینجا این که ممکن زمانی که به دلیل گسترش گره های تکراری ممکنه از دست بدیم را در نظر نگرفتیم .

در برخی از مسائل این اتفاق رخ نمی دهد چرا که برای هر حالت تنها یک مسیر وجود دارد

ولی در مورد بعضی از مسائل حالت تکراری غیر قابل اجتناب است (مسئله هایی که دارای اقدامات معکوس پذیری هستند مانند مسائل مسیریابی)

درخت های جستجوی این مسائل نامتناهی هستند اما در صورتی که برخی از حالت های تکراری را حذف کنیم میتوانیم درخت جستجو را به حالت متناهی برسانیم و فقط قسمتی از آن درخت را تولید کنیم که گراف فضای حالت را گسترش میدهد.

اگر حالت های تکراری شناسایی نشوند ممکن مسئله قابل حل، غیرقابل حل بنظر برسد

الگوریتمی که گذشته خود را فراموش کند، محنوم به تکرار آن است : اگر الگوریتمی تمام حالت هایی که از آن گذشته را نگه دارد آنگاه میشه گفت که این الگوریتم گراف فضای حالت را مستقیما کاوش میکند

ما الگوریتم tree-search را حال به طوری تغییر میدهم که ساختار داده closed داشته باشد تا تمام گره های گسترش یافته را ذخیره کند، اگر گره ای درون این ساختار داده باشد گسترش یافته نمیشود

به این الگوریتم جدید graph-search میگویند، زمان هایی که حالت تکراری بسیار زیاد داریم این الگوریتم بسیار موثر تر از tree-search می باشد

بهینگی توی جستجوی گراف کار سختیه ، گفتیم که اگر به دو حالت برسه که یکی جدید و یکی تکراری همیشه سراغ جدید میریم اینجوری اگه که مسیر جدیدمون به اون گره کوتاه تر باشه ممکنه که این الگوریتم یک مسیر مناسب را از دست بدهد

البته توی جستجو با هزینه یکنواخت و جستجوی اول سطح این اتفاق نمیوفته (این دو راهبرد بهینه جستجوی گراف هستند)

یا توی جستجوی عمیق شونده تکراری ممکنه قبل رسیدن به مسیر بهینه ، مسیر نیمه بهینه ای را دنبال کند در نتیجه این گراف باید بررسی کند که آیا مسیر تازه پیدا شده از مسیر اصلی بهتره

با استفاده از فهرست **closed** ، جستجوی اول عمق و عمیق شونده دیگر نیازی به فضای خطی ندارند از آنجایی که الگوریتم جستجوی گراف تمام گره ها را در حافظه نگه می دارد انجام بعضی جستجو ها به دلیل محدودیت حافظه عملی نیست.

جستجو با اطلاعات ناقص

تا قبل از این فرض می کردیم که محیط کاملاً رویت پذیر و معین است و کارگزار می داند که اثر هر اقدام چیست و می داند بعد از هر حالت به چه حالتی منجر میشود ، اما اگر دانش ناقص باشد چی ؟ انواع مختلف نقص سه حالت دارد

مسائل بدون حسگر (مسائل انطباقی)

اگر کارگزار هیچ حس گری نداشته باشد، آنگاه ممکن است در یکی از چندحالت اولیه ممکن باشد و در نتیجه اقدام ممکن است به یکی از چند حالت پسین منجر شود ، این مسائل فرض می شود از تمامی تأثیرات اقداماتش خبر دارد

در زمانی که دنیا کاملاً رویت پذیر نیست حتی اگر ندانیم که ابتدا در چه حالتی هستیم انجام یکسری از حرکت ها ، دنیا را مجبور می کند به حالت خاصی برود

کارگزار باید بجای یک حالت، درباره مجموعه حالتهایی که ممکن است در آن ها قرار گیرد استدلال کند. به چنین مجموعه ای **مجموعه حالت** میگویند

توی این مسائل به جای فضای حالت فیزیکی ، در فضای حالت باور جستجو می کنیم . هر حالت باور با هر اقدام به یک حالت باور دیگر ارتباط می دهد

مسائل اقتضایی

اگر محیط نیمه رویت پذیر باشد یا اقدامات غیر قطعی باشند، آنگاه پس از انجام هر اقدام اطلاعات جدیدی فراهم می کند.

اگر این عدم قطعیت ناشی از اقدامات یک کارگزار دیگه باشد آن مسئله ، یک مسئله **تخاصمی** است

هنگامی که محیط به گونه ایست که کارگزار میتواند پس از اقدام، اطلاعات جدیدی را از طریق حسگر هایش بدست آورد، کارگزار با مسئله اقتضایی روبه روست، راه حل این مسائل معمولاً به صورت یک درخت است، هر شاخه براساس ادراکاتی که تا آن نقطه درخت دریافت شده است

با بررسی فضای حالت باور، معلوم می‌شود که هیچ رشته اقدام ثابتی وجود ندارد که موفقیت را تضمین کند. الگوریتم های مسائل اقتضایی، پیچیده تر از الگوریتم های جستجوی استاندارد است.

مسائل کاوشی

هنگامی که حالتها و اقدامات محیط ناشناخته است، کارگزار باید برای کاوش اقدام کند، این مسائل را میشه حدنهایی مسائل اقتضایی دانست .